

Resource Accounting of Shared IT Resources in Multi-Tenant Clouds

Byung Chul Tak, Youngjin Kwon, and Bhuvan Urgaonkar, *Senior Member, IEEE*

Abstract—In today's IT platforms, the capability to accurately account overall resource usage among applications is crucial for variety of management actions (e.g., capacity planning, dynamic resource reallocation and/or load balancing). However, in the environments where small number of shared services cater to a large number of distinct entities' requests, resource accounting becomes significantly challenging. First, the overall resource consumption at the shared service is the aggregate of the resource consumption for multiple remote entities whose identities are not visible to the shared service. Second, even if such information becomes available, common monitoring tools (e.g., top, iostat) are unable to deliver accurate break-down of resource consumption since sharing occurs at sub-instance level (i.e., service instances are not exclusive). We study inherent challenges of performing resource accounting of shared resource. We compare two nonintrusive approaches having different balance between local monitoring and collective inference - (i) *LR* that uses easily-available tools which provide aggregate measurement and applying well-known linear regression as inference, and (ii) *Rameter* that puts more emphasis on gathering fine-grained per-thread information from within the hypervisor and applying light inference on the data. Evaluation shows that *Rameter* offers less than 1% error in accounting whereas *LR*'s error fluctuates between 5-150%.

Index Terms—Cloud computing, distributed system, and resource management

1 INTRODUCTION

ACHIEVING operational excellence on modern IT (Information Technology) platforms continues to become ever more challenging. The complexity of IT platforms keeps growing with emergent paradigms such as SDN (Software Defined Network). Furthermore, recent "Big Data" trends are necessitating increase of platform scales to unprecedented degrees. Today's management capabilities struggle to keep up with the increase of such complexity and scale [1], [2]. A large portion of this complexity stems from the sharing and consolidation of computing resources. Increasingly, IT platforms consolidate multiple S/W applications on a shared set of hardware equipment for reasons of cost-efficiency or organizational necessity. Such consolidation and sharing occur in a wide variety of platforms such as public clouds, private clouds, and even medium/small-scale data centers or clusters in enterprises or labs running multiple applications. Popular cloud-based shared services such as key-value stores (e.g., Amazon SimpleDB [3]) and relational database services (e.g., SQL Azure [4]) exemplify this trend. Considering a broader context, sharing also occurs for S/W, non-IT infrastructure resources such as cooling and power, and even personnel.

Clear understanding of how sharing happens for various resources can provide significant advantages in numerous

management scenarios. For one thing, it can be crucial for performance management. Suboptimal or untimely reaction to the current resource sharing state may adversely impact business operations. For instance, a load imbalance towards one of the replicas in a shared database VM may impact negatively the end-to-end delay of all the services that rely on it. Many industry data support that revenue is highly sensitive to even sub-second increase of delays [5]. Such understanding also helps administrators get answers to questions that are difficult to address otherwise. E.g., which app's request is triggering sudden burst of CPU saturations in one of the key-value storage servers deep down in the service pipeline? To which replica should I redirect such workloads so that overall resource utilization stays within a safe range? Is it also possible to apply resource capping so that fairness is maintained for the end users, and on what basis can we charge the user for the resource usage of the target server?

Proactively taking actions for resource usage adjustment or resource usage policy enforcement require accurate measurements of them. However, ascertainment of accurate resource usage information (the activity we refer to as *resource accounting* in this paper) is challenging within systems consisting of multiple servers running heterogeneous distributed applications. First, in many cases there are no clear indicators that tell us to which remote component or entity the resource usage should be attributed at a given time. For instance, at the database node of a multi-tier e-commerce application that is shared by several unrelated group of end-users, there is no explicit information about whose initial requests triggered for given currently issued database queries are unless the S/W stacks at each tier is modified to carry request identifiers. Such condition poses difficulties in managing the resource utilization of the database server by means of admission control at the user-facing component.

- B. C. Tak is with the IBM T. J. Watson Research Center.
- Y. Kwon is with the Computer Science Department, University of Texas at Austin, Austin, TX.
- B. Urgaonkar is with the Department of CSE, The Pennsylvania State University, PA.

Manuscript received 30 Jan. 2015; revised 8 June 2015; accepted 1 July 2015. Date of publication 8 July 2015; date of current version 7 Apr. 2017. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below. Digital Object Identifier no. 10.1109/TSC.2015.2453980

Another difficulty is that, even with such knowledge, resource attribution is non-trivial due to the followings. Simple utilization information of processes or threads (such as `top`, `iostat`) may be insufficient because those numbers are the aggregate resource utilization caused by multiple remote entities. This is due to the granularity of resource principals being as fine as threads and even the bindings between the remote resource consuming entities and such resource principals could change dynamically over time. This implies that we need to further break down the monitored values using inference techniques. Other example is the write activity of disk I/O in which OS kernel combines multiple writes and issues them much later in time non-deterministically. These challenges are intensified especially in today's complex service architectures in which many services are built on top of other heterogeneous services via interface such as REST APIs (Representational State Transfer).

Based on our study, we believe that current state-of-the-art needs to be improved to deliver aforementioned resource accounting capabilities for improving cloud operations and management. Toward this end, we have designed and implemented the resource accounting technique, called *Rameter*. *Rameter* consists of two modules—(i) distributed request (message) causality tracking module running within the hypervisor, and (ii) resource usage accounting module that gathers thread scheduling, spawning and I/O activities. Our distributed request tracking technique allows us to first determine the ownership of request messages at any VM or server nodes without the need to modify the application codes. Once the ownership is determined, the resource accounting module observes fine-grained thread level events from the hypervisor to further break down the CPU and I/O usages.

Compared to our *Rameter*, some of the existing techniques rely on modifying (all or part of) the OS or middleware stacks to enrich the collected data [6], [7]. Often, they specialize on one specific component and implement the accounting and/or controlling functions by modifying source code [8], [9]. However, such instrumentation-based approaches are viable only to the organization who builds their own S/W stacks or to the relatively small environment such as embedded systems. And, even with some instrumentation, gathered monitoring data may not contain enough information for the purpose of accurate resource accounting. In response to such limitations, we pursue the goal of building effective resource accounting technique that is (i) focusing on obtaining fine-grained per-thread resource usage information, (ii) generally applicable to existing environments, (iii) accurate enough for any type of resources, including both IT and non-IT resources, and (iv) flexible enough to accommodate a wide-range of resource management policies. As an initial step, we focus on IT resource such as CPU, I/O and network in a virtualized environment. Note that our study is *not* about proposing the adoption of more detailed billing information to the cloud users. We address the resource management problem from the perspective of cloud service providers with the goal of improving the cloud management operations.

We study the resource accounting problem and investigate solutions that addresses all challenges identified above. Towards finding the right solution we build and compare two approaches that differ in where they focus more among

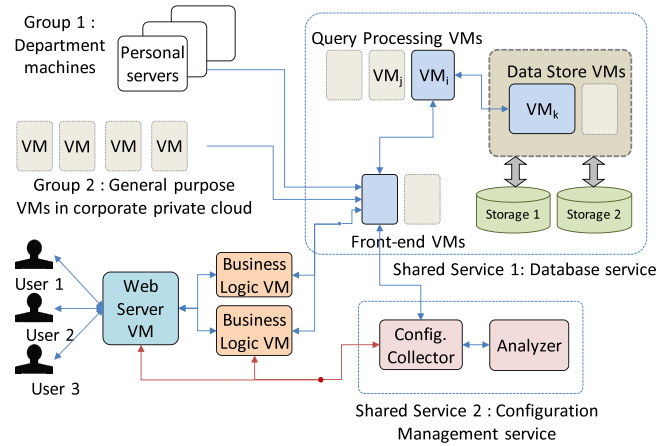


Fig. 1. Problem model. CE (Chargeable Entity) is defined to be any entity or group of entities that sends requests directly or indirectly.

data collection and inferencing during the overall resource accounting. One central concern in our design is to achieve generality by avoiding application modification. Specifically, we make the following key contributions. First, we formulate the problem of resource accounting for shared servers running distributed applications, and introduce the design of *Rameter*, capable of providing accurate resource usage information of shared services. To the best of our knowledge, our design goes beyond the state-of-the-art by being the accounting solution that is both: (i) implemented within privileged code (VMM in our case) with no need for application modification, and (ii) capable of accounting the usage of shared services. Second, using a mix of synthetic and real-world shared services, we present the comparison of two approaches—*LR* that uses easily-available system utilities which provide aggregate measurement data and applying well-known linear regression as inference, and (ii) *Rameter* that puts more emphasis on gathering fine-grained information from within the hypervisor and applying light inference on the data. Third, we demonstrate the usefulness of *Rameter's* resource accounting information by applying it to online resource control (specifically via throttling) using the scenario of enforcing the SLA requirements for RUBiS application under heavy workloads. Such capability of online resource control for shared services is achievable only when fine-grained thread-level information is available, which aggregate monitoring data cannot provide.

The rest of this paper is organized as follows. Section 2 provides some background knowledge and explains what makes the resource accounting problem challenging. In Section 3, we define our problem. In Section 4, we identify key design requirements. Descriptions of our solution are given in Section 5, and it is followed by implementation details in Section 6. In Section 7, we present our experimental evaluation. Then, we provide related work in Section 8. Finally, in Section 9, we present concluding remarks.

2 BACKGROUND AND CHALLENGES

2.1 Problem Model, Terminology and Scope

We use Fig. 1 that shows a representative platform that we use to drive our discussion. It illustrates various key entities and the relationships between them. We refer to a platform

user or their application whose resource usage must be separately tracked and accounted as a *chargeable entity* (CE). The concept of CE is not limited to subscribers of software services. We define it in a broader sense to recognize groups of computing equipment as CEs as well. Fig. 1 shows several types of such CEs—Group 1, 2, User 1, 2, and 3. However, if necessary, any other entities such as “Business Logic VM” or individual VMs in Group 2 can be treated as separate CEs. We assume the virtualized environment in which CEs’ software components run within virtual machines (VM).

Also shown in the figure are example shared services that the platform offers to its CEs—a database service and a configuration management service. These shared services themselves have multiple components that span across several VMs or physical servers. Internally the database service maintains functionally separate group of VMs for load balancing. Whereas its front-end tier communicates directly with the chargeable entities, its back-end components are exercised indirectly, i.e., via requests made to the front-end tier. The arrow from the configuration management service to the database service represents the fact that services may build on top of other services. Note that each CE does not exclusively own its database process. One database instance (in our example, two front-ends and three query processing VMs for one instance) may be shared by an unrelated group of tenants.¹ Shared services in our model are frequently found in Platform-as-a-Service (PaaS) type cloud where customers pay for the usage of services rather than owning exclusive service instances.

Unlike for ‘Web Server VM’ or ‘Business Logic VM’ where an accounting-capable VMM (e.g., using an existing accounting solution such as resource containers [10]) could associate the VMs with appropriate CEs, existing solutions cannot be directly adapted for accounting within servers that belong to shared services where the VMM-visible resource principals do not have a fixed association with any CE. Additionally, since the back-end tier of the database service is only exercised by the CEs *indirectly*, i.e., via work generated during processing of requests that are made by CEs to the front-end, additional thought is needed to identify what portion of its resource usage should be attributed to which CE. Consequently, a resource may be used by a CE either *directly* (e.g., resources on server VMs exclusively assigned for the software owned by them) or *indirectly* via a shared service (e.g., resources on servers hosting the database-as-a-service, within the SaaS (Software-as-a-Service) database, the shared network, or the SAN).

Recently a number of data center resource management solutions (e.g., DCOS [11], [12], [13], fos [14], and OpenStack [15]) have emerged all of which require solutions for keeping track of and accounting the overall usage of shared resources, typically at the granularity of a container or VM. We view our work as highly complementary to these solutions in that our technique can be used in these solutions for even finer task granularity accounting.

1. The term, tenant, is the equivalent of the CE in database terminology. However, it typically refers to the direct subscribers of the database service itself, not of the entire platform as how CE is defined.

In resource management, *fairness* can be defined in several ways. They could be either strictly in terms of the amount of resource usage, or in relation to how much user is willing to pay for the service quality. In any case, accurate resource accounting is the basis for the enforcement of the fairness. The scope in this work is to build supporting mechanism to enable the enforcement of various fairness policies. Extending Rameter to enforce the fairness in terms of the service quality and payment is our future plan.

2.2 Real-World Examples of Shared Services

It is not hard to find the shared services in both the public clouds (especially in PaaS) and/or non-cloud environments that conform well to our model. Valid example should exhibit the behavior in which the resource sharing occurs at the instance-level (i.e., process). If separate instance of VMs or processes serves different CEs, then managing the resource usage becomes significantly easier. However, instance-level sharing is becoming more common because it provides better scalability and resource efficiency. Real-world examples of shared services that conform to our model are listed below.

SQL Azure. The first example of a shared service is the Microsoft SQL Azure [4], the multi-tenant SaaS database service. In SQL Azure, tenants see only their own database spaces, but they physically share the underlying database server instance with other tenants [4]. Windows Azure also offers three different types of shared storage services—Blob, Table, and Queue—intended for different purpose. Tenants are given separate database spaces, but the database instances are shared [4].

Force.com. It is a (Platform-as-a-Service version of the *Salesforce.com*, which is a SaaS cloud service specializing in providing an online CRM (Customer Relationship Management) solution. By design, *Force.com* adopts an architecture in which all the users share a single software instance and the set of H/W. [16]. This architecture is chosen to increase the manageability.

SimpleDB. An example of the shared service can also be found in AWS (Amazon Web Services) cloud. Amazon offers non-relational key-value data store service, called SimpleDB [3]. Subscribers are charged by the machine utilization, data transfer size incurred by user requests and the total storage space used. Especially, SimpleDB measures the CPU utilization at the individual request level. CPU utilization consumption is estimated based on the amount of data size. Although system architecture is not publically available, it can be inferred that SimpleDB adopts some form of resource accounting mechanism.

Bigtable. Bigtable [17] is Google’s proprietary database built with massive scalability in mind. Bigtable service builds on top of two other services—Chubby [18] and GFS (Google File System) [19]. Chubby is a distributed lock service manager that provides coarse-grained synchronization on shared resources. Bigtable uses Chubby for maintaining a master Bigtable server and for storing the metadata for data locations. GFS is used by the Bigtable to store logs and data files. Bigtable internally serves the needs of multiple groups in Google. Each group is given isolated table space, but they share the Bigtable instance similar to SQL Azure.

2.3 Challenges

2.3.1 Lack of Direct Indicators

A key difficulty arises due to lack of direct indicators of CEs responsible for the currently in-progress resource activities at the servers, especially in shared services. Unlike application-owned software, a shared service *may only be exercised by a CE indirectly*, making it more difficult to ascertain this relationship. In Fig. 1, the query processing VMs of the database service (Shared Service 1) is merely invoked indirectly by the CEs through the front-ends, oblivious of who it is working for. Likewise for the front-ends, they are only able to identify whether the request is from Group 1,2, Business Logic VMs or the Config. collector. But, it is hard to identify further whether the requests coming from the Business Logic VMs are from User 1, 2 or 3. One approach for inferring such relationship may be to instrument the messaging S/W to inject identifiers, which is not generally applicable and may be prohibitive.

How accurately the local monitoring on server can identify such indirect usage depends on its accuracy in recognizing the underlying *causation* (i.e., some activities of a CE caused certain activities of a component which consumed some resources). When a CE is only “one hop away” from the component, the presence of direct communication between them can yield this causation information. However, identifying causation becomes trickier when the component is “more than one hop away” from the CE. Solving this problem, in general, requires some form of statistical inference based on probabilistic models to capture this causation, and closely related examples can be seen in some work [6], [20].

2.3.2 Mismatch of Resource Principals and CEs

If application-owned S/W components are contained within resource principals that are likely to be easily identifiable by underlying resource management software (e.g., the virtual machine monitors (VMMs)), this implies an existing local accounting solution such as resource containers can be easily used by this management software to associate these resource principals with the corresponding CE.² On the other hand, the shared service’s software design and configuration *may not be amenable to easy adaption of such existing solutions for local accounting*. For example, the data store component in Fig. 1 multiplexes the resources assigned to its internal schedulable entities (e.g., threads) in highly application-specific (and possibly unknown) ways among the activities it carries out on behalf of CEs, rendering a solution such as resource containers difficult to adapt.

3 PROBLEM DEFINITION

Let us denote the chargeable entity by $c \in \mathcal{C}$ where $\mathcal{C}$ is the set of all CEs, and a server by s . Server s is one of the servers in a shared service group $\mathcal{S}$ for which we are interested in performing the resource accounting. For server s we

TABLE 1
Summary of Symbols

Symbol	Description
c	Chargeable entity, CE
$\mathcal{C}$	Total set of CE
s	Server
$\mathcal{S}$	Group of servers comprising a shared service
r	Resource type (e.g. CPU, Disk I/O and/or Network I/O)
n	Number of resource types we consider
$\mathcal{U}_c^{r,s}(t)$	Resource utilization time-series of type r at s attributable to c
$\mathcal{U}_c^{r,s}(t)$	Resource utilization time-series of type r at s attributable to c
$\mathcal{V}^{r,s}(t)$	Unaccountable resource utilization time-series of type r at s
$\mathcal{W}^{r,s}(t)$	Resource idle time-series of type r at s

produce n time-series, one per resource type r at a measurement granularity Δ . We denote as $\mathcal{U}_c^{r,s}(t)$ the utilization measurement for resource r at server s during the t th time slot. Its range is $0 \leq \mathcal{U}_c^{r,s}(t) \leq 1$ to indicate proportions. Table 1 summarizes the symbols and their meanings.

The goal of *resource accounting* is to infer, for each CE c at s , the contribution of c to the utilization, $\mathcal{U}_c^{r,s}(t)$. In addition, we are interested in finding out the proportion that is not attributable to any c , denoted as $\mathcal{V}^{r,s}(t)$. That is, we want to find $\mathcal{U}_c^{r,s}(t)$ and $\mathcal{V}^{r,s}(t)$ such that

$$\sum_{c \in \mathcal{C}} \mathcal{U}_c^{r,s}(t) + \mathcal{V}^{r,s}(t) = \mathcal{U}^{r,s}(t). \quad (1)$$

As a corollary, if we let $\mathcal{W}^{r,s}(t)$ be the proportion of unused (i.e., idle) resource at time t at server s , it would follow that $\sum_{c \in \mathcal{C}} \mathcal{U}_c^{r,s}(t) + \mathcal{V}^{r,s}(t) + \mathcal{W}^{r,s}(t) = 1$. Similarly, $\sum_{t_i \leq t \leq t_j} \sum_{s \in \mathcal{S}} \mathcal{U}_c^{r,s}(t)$ would represent the usage of resource type r by c in the entire shared service group $\mathcal{S}$ during the time period of $t_i \leq t \leq t_j$.

4 DESIGN PRINCIPLES

Any accounting solution that forms the basis for the decision of resource controlling actions must have two elements: (i) *local monitoring* and (ii) *collective inference*. We use the phrase “local monitoring” to refer to facilities within each server that record events and statistics pertaining to the resource usage of (or on behalf of) each CE. E.g., in resource containers, local monitoring is carried out by the server operating system that is modified to identify resource allocation/scheduling events (e.g., when threads are scheduled/descheduled on the CPU) and using this information to charge their usage to appropriate containers [10]. The phrase “collective inference” refers to the functionality needed to combine the pieces of information offered by local monitoring to create a correct overall picture of accounting. Since resource containers are only concerned with a single server, collective inference is trivially realized from the monitored data. Distributed resource containers must address a more complicated version of collective inference, and it does this by augmenting the locally monitored data within each server with the identity of the distributed container

2. In fact, this is the essential idea behind distributed resource containers [21]: individual servers use resource containers for local accounting and the network stacks within server operating systems are modified to embed tokens within messages sent to/by components that uniquely identify their CEs.

(carried within messages exchanged between container components) that they correspond to [21].

As argued in Section 3, both local monitoring and collective inference need to be reconsidered for servers running shared services. There exist a large number of techniques and tools for local monitoring that one could choose from. These existing techniques span a wide spectrum of the “level of detail” they offer at the cost of generality, application intrusiveness, and overheads posed. At one end of this spectrum are techniques that can instrument user-space and OS/VMM code to create a very detailed record of a shared service’s resource usage that contains sufficient information for collective inference [22]. At the other end of the spectrum are CE-oblivious resource usage reporting tools that rely on information available within the server’s OS and VMM. E.g., *top*, and *iostat*.

As we will empirically show in Section 7, collective inference that relies on data offered by these tools can have significant inaccuracies in accounting. Furthermore, as we will find, such inference can be extremely sensitive to a variety of system properties and environmental conditions, an undesirable feature. Although our analysis of inference using existing tools will be based on a specific inference technique, we argue that the root cause of these inaccuracies is the inadequacy of information contained in the monitoring information offered by these tools, and even more sophisticated inference techniques relying on such information would falter.

Generally speaking, collective inference is a statistical learning problem that must derive models that can meaningfully tie together the data provided by local monitors, possibly filling in any gaps or discrepancies within these data. The efficacy of such inference crucially depends upon the resource usage phenomena collected by local monitoring elements. Existing monitoring tools that are not application-intrusive have been designed for information collection at the granularity of OS/VMM-relevant abstractions (e.g., threads, TCP connections) that may not coincide with the needs of our accounting. Consequently we identify the following design principle that underlies our accounting solution: *our local monitoring must explicitly capture information pertaining to resource usage on behalf of CEs to allow accurate accounting by our collective inference.*

5 RAMETER: RESOURCE ACCOUNTING FRAMEWORK

Guided by the design principles stated in Section 4 regarding the consequence of choosing the local monitoring technique and the collective inference algorithms, we develop a resource accounting technique, called *Rameter*, that possesses following characteristics. First, our solution does not require modifications to applications and/or middleware. Second, the granularity of data collected from the local monitoring is at the thread-level. In this section we describe the general ideas underlying *Rameter*.

5.1 Local Monitoring

The key aspect of local monitoring that we need to perform is identifying and recording information about resource principals and scheduling events of interest. Recall from

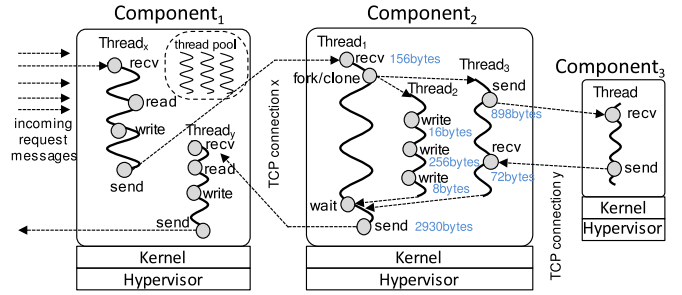


Fig. 2. Illustration of solution concept using an end-to-end flow of one of incoming requests.

Section 2.3, that real challenges in answering this question arise when CE *c* uses resources on server *s* indirectly, i.e., when a shared service running on *s* consumes resources on behalf of *c*.

5.1.1 Identifying the Causation and Interval

To recognize a CE *c* that is more than one hop away, we have devised a technique that could track the causal relationship between incoming and outgoing messages in a component. Let us use Fig. 2 to explain its key idea. The figure illustrates an example end-to-end flow of a user request across distributed components. Although it shows the message flow of a single request across components, it should be noted that multiple requests are being processed concurrently. Starting from the first *recv* system call by the Thread_x, the initial request originated from a CE produces a flow of causally related system call events until a reply is sent back to the CE. Between these *recv* and *send*, we can track the sequence of system call events across components using the following rules. First, within the component, the causality is carried by the thread. Second, when the thread sends a message to other components, the causality moves to a thread in the receiving component. In order to apply these rules, we collect the system call events related to the network activities. Additionally, it records the thread IDs and socket tuple information (i.e., IP and port number of sender and receiver components) for each events. Thread ID enables us to connect the activities that belong to the same threads, and the socket tuple allows us to connect *send* and *recv* pairs across components.

However, resource accounting requires detecting more events than only those related to network activities. First, creation of new threads must be tracked. Processing one request may involve spawning of multiple threads or processes and each may consume system resources in a different way. Component₂ of Fig. 2 illustrates such example. Suppose that Thread₁ spawns two threads via *fork* (or *clone*) system calls upon receiving a request message. Thread₂ incurs multiple disk I/Os, whereas Thread₃ interacts with another component. In this example, the correct CPU consumption for processing of the request message given to the Component₂ is the sum of CPU cycles consumed by all three threads. Second, the return value of system calls are needed. Return values of system calls indicate the amount of data handled by the system call. In order to determine the size of I/Os performed, we need to extract those return values. Another requirement for the capability of resource control is the online causality tracking within

the hypervisor. We provide implementation details specific to our environment in Section 6 and discuss other techniques for different environments.

5.1.2 Thread Scheduling Events

This aspect of local monitoring is concerned with detecting the exact moment of thread context switch and collecting information about when the thread begins to use a resource on behalf of which CE as well as when it stops doing so. The local monitoring must record such information solely based on what the hypervisor can observe about the resource principals on that server, and the events corresponding to their scheduling.

For a server s indirectly used by a CE c (i.e., running a shared service component s exercised by c), we need to identify CPU (de-)scheduling events *within* the software of s that correspond to durations for which s was using the CPU on behalf of c . Identification of any I/O activities initiated during these same periods allows for accounting I/O bandwidth usage by s on behalf of c . For example, Thread₁ of Component₂ in Fig. 2 may experience three different types of scheduling events between `recv` and `send` - VM, process and thread (de-)scheduling events. Specific implementation details we have employed to achieve these are provided in Section 6.

5.2 Collective Inference

Given the extensive information that our local monitoring gathers, collective inference for accounting CPU, network and disk I/O bandwidth essentially boils down to aggregation of the resource usage information collected by various local monitoring units. Depending on the type and ownership of resources, different methods should be employed.

Definition. We use the term, *segment*, as referring to the duration of time starting from the arrival of a request message by `recv` until sending of another message by `send` within a component. Interacting components of those `recv` and `send` are not necessarily the same.

5.2.1 CPU

Basic idea of CPU accounting is to measure the time difference between `recv` and `send` events of each request message (i.e., segment), and to add the difference to the corresponding CE's variable that holds the accumulated CPU usage. In addition, we subtract the time when the thread was descheduled using the information about scheduling events collected as a part of local monitoring. The time is measured by RDTSC instruction, which contains the number of cycle counts since boot. Sometimes we can encounter a segment of thread execution that does not start with `recv`. These CPU consumptions can be due to background (periodic) activities of OS or daemons. These segments that cannot be labeled with appropriate CE is treated as 'unaccountable' (See Figs. 7b and 13a for example). The 'unaccountable' quantities tell us the possible range of errors in CPU accounting. It is our future goal to extend accountings to them and reduce errors.

5.2.2 Network and Disk I/O

All the network related activities within the segment is accounted to the CE currently associated with the thread.

We observe the occurrences and return values of system calls such as `recv`, `send`, `recvfrom`, and `sendto`,³ and accumulate their return values to come up with network bandwidth usage. Note that this quantity does not include the bandwidth consumption due to protocol-specific overheads such as retransmissions and various header/trailer portions added across the protocol stacks.

Disk I/O accounting is done similarly as the network I/O (i.e., by tracking the return values of `read` and `write` system calls). However, the resource usage we collect is different from the actual disk bandwidth consumption observed by the storage device. This is due to nondeterminism introduced by the page cache and block I/O optimization mechanisms by the kernel. If exact accounting of physical disk bandwidth is required, appropriate inference technique must be employed on top of the accounting information we provide.

5.3 Limitations

We describe some of known limitations of current Rameter. First, Rameter is currently not able to account memory usage due to overhead in tracking individual memory accesses. Detecting the memory access by inducing page faults is one option, but high performance impact renders it impractical. Second, there are some gaps between reported Rameter's I/O accounting results and the actual 'physical' resource usage as pointed out in previous subsection. This is because some of the I/Os detected by Rameter may not translate to actual physical I/Os due to caching. Locks held by shared resources may also cause Rameter to produce misleading results. Resources consumed by a thread while busy-waiting on locks are counted as resource consumption by Rameter when there is no actual progress of work being made. Third, Rameter does not try to account for the effects of interleaved workloads from multiple CEs. For example, CE c_1 may end up issuing more disk I/O requests because the workload from concurrently running c_2 flushes c_1 's data from the internal cache. However, Rameter still focuses on faithfully reporting what has happened, rather than who caused it. Such analysis requires additional inferences and it is left as a future work.

6 IMPLEMENTATION

In this section, we describe implementation details specific to our environment. Our virtualization environment is based on Xen 3.1.4, 32-bit para-virtualization. Required modifications are mostly for enabling the local monitoring techniques as described in Section 5.1. Collective inference is independent of environment specifics since it is a processing of data gathered from the local monitoring step. In order to realize the local monitoring of Section 5.1, we need to implement three capabilities—system call interception, capturing of system call return values, and detection of scheduling events. Techniques described below are dependent on our platform's specifications, and different set of techniques may need to be implemented to achieve similar capabilities in other environment.

3. The `read` and `write` syscalls are used for both network and disk I/Os.

6.1 System Call Interception

For intercepting the entry point of the system calls invoked by user applications in guest VMs, we modify the Xen hypervisor in the following way. In our para-virtualized Xen environment, system calls use INT 80 h mechanism. To intercept the system calls, we first add an system call handler entry in the exception table. Then, we register a system gate for interrupt 80 h to the IDT (interrupt descriptor table). Whenever a guest VM fires a system call, the software interrupt 80 h is triggered. CPU, then, looks up the IDT and jumps to the system gate we installed. Within it, the address of the custom handler is searched from the exception table and, finally, the control comes to the custom handler.

More common system call mechanism is SYSENTER/SYSEXIT. INT 80 h is mainly used in the para-virtualization since SYSENTER/SYSEXIT is hardwired in the processor to assume that the kernel resides in ring 0, whereas in para-virtualization, the guest VM's kernel is in ring 1. In full-virtualization, SYSENTER mechanism is being used because the guest kernel runs in ring 0. Although we didn't need to handle this, system call based on SYSENTER can still be intercepted using techniques described in Ether [23].

6.2 Capturing System Call Return Values

At the entry point of the system calls, we have access to information such as file descriptor number, buffer address, buffer size and buffer contents. However, the return value is available only at the exit point of the system call. There are several techniques for intercepting the exit point. One way is to leverage the IRET hypercall handler built into the Xen hypervisor. In 64-bit Xen, they are invoked transparently, whereas in 32-bit mode, it is enabled only when VM86 mode is set, which is the backward compatibility mode that allows real mode instructions to execute in the protected mode. For 32-bit Xen, we need to modify one `jnz` instruction to `jmp` in `arch/i386/kernel/entry-xen.S` to force the IRET hypercall. Another method is to use the page protection mechanism. At the system call entry point, we perform two tasks. First, we turn off the `PAGE_PRESENT` bit of the page table entry that holds the user stack page address. Second, we record the address of kernel stack which is available in one of the Xen-defined variables. When the system call exits, it will attempt to write the return value from EAX to the user stack. This will trap and, when the Xen hypervisor gains the control, the EAX value can be read from the kernel stack. We have tested these mechanisms and confirmed that they were able to retrieve the return values. However, for the convenience in the experiment, we have inserted custom hypercalls to the exit point of the guest kernel system calls and directly delivered the return values to the hypervisor.

6.3 Detection of Scheduling Events

There are three types of scheduling events we need to detect—VM scheduling, process scheduling and thread scheduling. Detection of VM scheduling events is trivial since it is done by the hypervisor. For other scheduling events, the scheduling object in Linux is the light weight process, equivalent to the thread. Thus, detection of thread scheduling event covers the process scheduling. In our

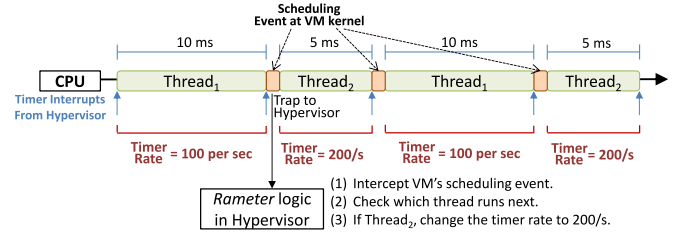


Fig. 3. Hypervisor-based CPU resource control mechanism by manipulation of VM-bound timer interrupts.

32-bit Xen para-virtualization environment, whenever stack switching happens, it traps to the Xen hypervisor (`do_stack_switch` function in `mm.c`). We have added here the codes for recording thread scheduling events using Xen-provided TRACE macro.

6.4 CPU Resource Control from Hypervisor

In order to demonstrate the effectiveness of Rameter in the evaluation, we employ a hypervisor-based CPU resource control mechanism to a reactive resource control scenario (see Section 7.3.1). It takes Rameter's resource accounting results as its input and performs targeted throttling of the CPU consumption caused by the CE that is overusing the CPU at the shared server. This is achieved by manipulating VM-bound timer interrupts from within the hypervisor. It is non-intrusive and transparent to the guest VMs. Currently this technique is limited to the CPU resource only. We describe key principles and implementation details in this subsection.

Rameter logic in the hypervisor maintains current status of per-CE CPU resource consumption of the target VM. When it detects that one of the CE starts to exceed the CPU consumption defined by a policy, it initiates the resource control technique, whose principle is described in Fig. 3. It shows an example thread scheduling sequence at a CPU that is assigned to the target VM, where two threads, thread₁ and thread₂, alternate. Let us assume Rameter has already determined that thread₁ and thread₂ are bound to CE₁ and CE₂, respectively. Rameter aims to reduce the CPU usage of CE₂, and, thus, thread₂ is the subject of throttling in this case. Whenever Rameter in the hypervisor detects the scheduling event of the VM, it checks if next thread is thread₂. If so, Rameter increases the frequency of timer interrupts from 100/s to 200/s until next scheduling event fires. Since VM's notion of time is based on the timer interrupts from the hypervisor, faster timer causes the next scheduling to happen sooner. In the figure, thread₂ is descheduled only after 5 ms, although VM's OS kernels thinks 10 ms has passed. Timer rate is restored when thread₂ is descheduled.

VM scheduling events are captured by Rameter when thread context switch occurs. In Xen 3.1.4, all the context switching traps at `do_stack_switch()` in `xen/arch/x86/x86_32/mm.c`. Within that function we identify the thread by observing the kernel stack address given by `esp` parameter. If it matches the stack of target thread, we change the timer rate by assigning new rate as this: `current->periodic_period=MILLISECS(1)`. Later when this thread is descheduled, we switch back to `MILLISECS(10)` to restore the normal rate.

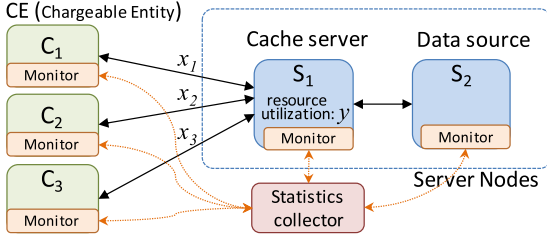


Fig. 4. Set-up of our synthetic shared service and the CEs exercising it, where each server here is a separate VM.

The principles of resource control described here can be readily applied to container environments as well. Monitoring and/or changing scheduling behavior in containers is generally easier than from the native hypervisors. Threads in containers are visible from the host in the same way it sees any other threads. It is also straightforward to change the scheduling quantum of targeted threads which is the basis of our resource control. Note that timer manipulation is no longer needed for containers because of that.

7 EVALUATION

In this section we evaluate the efficacy of Rameter and compare it against a baseline technique that uses commonly available statistics combined with linear regression as an inference method. First, we explore Rameter's accuracy using CEs and a shared service that are based on home-grown programs for which the "ground truth" can be found with confidence. Next, we employ Rameter for accounting the usage of two real-life applications used as shared services—a clustered MySQL database server and an HBase key-value store. Throughout, we compare Rameter against a baseline accounting technique called LR (described below) that relies upon readily available resource usage information available in today's servers.

7.1 Baseline Accounting Technique

Our baseline is based on the linear regression model. LR-based modeling technique is popular in modeling the resource usage due to its simplicity, interpretability and good conformance to linearity for many cases. Gupta et al. [24] have applied LR to predicting the CPU usage of a VM using the incoming network traffic volume as the input and have found that it performed well. LR is also shown to be effective in modeling and predicting the resource requirements due to virtualization overhead by Wood et al. [25]. Power resource usage modeling is another active domain of LR's application [26], [27]. All of the existing application LR tries to model the resource usage at the granularity of VM or individual server. Our key point of study is whether LR is still effective when modeling the resource usage of shared services by various CEs.

Our LR model relates the resource usage $y(t)$ of S_i (e.g. CPU utilization time-series) to the inbound network traffic volume $x_i(t)$ from each C_i . That is, we form the following equation for each interval t :

$$a_0 + a_1 \cdot x_1(t) + a_2 \cdot x_2(t) + \dots + a_n \cdot x_n(t) = y(t). \quad (2)$$

In applying this model, we assume that conditions for linear regression such as linearity between x and y ,

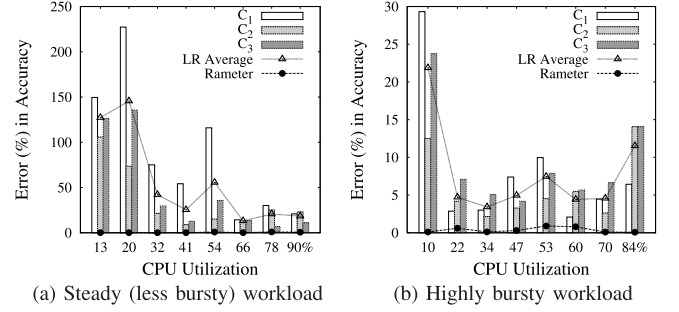


Fig. 5. Impact of workload burstiness on the accuracy by Rameter versus LR at S_1 in our synthetic shared service, and CPU utilization. We show error percentages for three CEs (C_1, C_2, C_3) with LR, and label their average as "LR Average." In all cases, Rameter offers less than 1 percent error.

non-existence of multi-collinearity among x , constant variance of errors holds. As will be shown, various system properties can violate such pre-conditions and lead to undesirable results. We solve it using the least squares method to obtain the coefficients ($a_0, a_1 \dots a_n$) for each interval t . Since each coefficient represents the contribution of each $x_i(t)$ to the resource usage, we treated them as proportions to divide the resource utilization $y(t)$, giving us the breakdown by each CE at S_i at time t . Since we wrote server codes, we were able to measure the exact amount of CPU resources consumed by each CE, which enabled us to calculate the accuracy of LR-based technique.

7.2 Accounting Accuracy for a Synthetic Service

7.2.1 Experimental Setup

Fig. 4 shows the design and configuration of a synthetic shared service we employ. We use a two-tiered design for the shared service with the front-end acting as a caching tier. This is a simple data store service in which the front-end simulates the data processing and the back-end the data storage. The front-end is configured to maintain a cache size of 300 KB. A user request specifies the data address and the data size in unit of KB. Each user request incurs CPU consumption to mimic the data processing overhead. Cache misses at the front-tier result in work generated at the back-end. Multiple clients send requests to the front-end during long-lasting sessions and correspond to our synthetic CEs. Statistics collector gathers CPU, network, and disk I/O utilizations for LR. LR equation is formed using x_i and y as marked in the figure. Separately, Xen hypervisor runs its own local monitoring. The reason we use the synthetic benchmark is because, having the source code, it allows us to engineer the exact amount of *true* resource consumption we want for determining the error of each technique we apply.

7.2.2 Effect of Bursty versus Non-Bursty Workload

Fig. 5 compares the effect of burstiness/variance in workload on the accuracy of CPU accounting at S_1 . Different values of the average request rate are imposed on the shared service by a group of three chargeable entities C_1, C_2, C_3 , (which create different CPU utilization levels at the server S_1). Eight levels of request rates are chosen for each CE ranging from 15 to 120 req/s at an increment of

15. We pick a “non-bursty” scenario where the requests are uniformly spaced in time, and a “bursty” scenario where the request inter-arrival times follow lognormal (0,1,0) distribution. We find that the efficacy of LR varies depending upon the extent of variation within the imposed workload. This is in line with results known in existing work that finds non-stationarity in workloads useful for certain kinds of prediction and modeling [28]. Intuitively, better accuracy is achieved with bursty workloads because the higher variety/dynamism in the input data supplies more information to LR; we expect this basic insight to apply to any statistical inference technique for accounting. For a less bursty workload, a large part of the input data may be redundant and not offer new information to an inference technique. On the other hand, by virtue of its direct measurement of relevant phenomena, Rameter is able to achieve accurate accounting that is robust to changes in such workload conditions.

7.2.3 Effect of Other Factors

We have also studied the effect of caching and buffering as well as varying number of chargeable entities. We find that caching and buffering—both valuable and prevalent performance enhancement techniques—affect the accuracy of accounting of a technique like LR. As is well-known in general, a cache within or in front of a service can destroy/distort correlations between its incoming request/traffic events and the workload imposed on its underlying server in complex ways. Buffering of requests/traffic can also have a similar effect by modifying the time lag between an event (e.g., the issuance of a request) and its cause (e.g., the actual servicing) in complicated ways. We have carried out experiments where we vary several factors affecting the degree and nature of caching within the front-tier of our shared service (see Fig. 4): request size (fixed or varying), read/write ratio (from 10:1 to 1:1), temporal locality (non-existent to very high), and the extent of common/overlapping content requested by the chargeable entities. Although graphs are omitted for the interest of space, we observe that caching factors described above also have various degree of impacts to the accuracy LR results. The error of LR-based resource accounting ranged from 10 to 60 percent with high variance whereas Rameter exhibited less than 1 percent error for all cases. Accuracy gains from the burstiness of the workloads can be easily offset if application happens to employ some form of caching structure internally.

7.2.4 Summary of Key Findings

To summarize, we find that the efficacy of LR relies upon both the quality of data it gathers as well as the presence/extent of correlation between its inputs and outputs. Even when accurate data can be obtained (as with our implementation of LR), several factors including (i) inherent workload properties (e.g., variance, temporal locality, intensity), (ii) system mechanisms and algorithms (e.g., caching or buffering), and (iii) environmental conditions (e.g., degree of resource interference from other S/W) might affect such correlation and affect the accuracy of the accounting technique. We find empirical evidence that, owing to its ability to directly measure

relevant phenomena accurately, Rameter is robust to such effects, and offers high-accuracy accounting information across a wide range of operating conditions.

7.3 Evaluation With Real-World Services

In this section, we report on the results of accounting the CPU and network bandwidth usage of two real-world applications—MySQL Cluster and HBase. They are both set up to be shared by multiple Chargeable Entities. MySQL Cluster represents the class of relational database services, and the HBase the class of key-value store services, both commonly found in modern cloud services. We compare the resource accounting capabilities of LR with our Rameter technique and present the case where LR makes an incorrect conclusion whereas Rameter remains robust. In general, we cannot expect to determine a real-world application’s actual resource usage on behalf of different chargeable entities without resorting to extensive application and OS instrumentation. Consequently, unlike for our synthetic shared service, we cannot obtain/present a direct comparison of the efficacy of our techniques, i.e., distance of the accounting information offered by Rameter versus that offered by LR from the “ground truth.” Therefore the focus of this section is not on the accuracy of the accounting results from both LR and Rameter. In actual systems management, this *rank order* can be more important than the accuracy of resource accounting since management algorithms often need to pick the victim to enforce actions to. If a resource accounting technique tells you the resource consumption quantity with certain level of error (although it may be unknown), it can be less problematic than the case in which it picks the wrong entity as the largest consumer of the resource. We demonstrate in this section that such cases do exist under LR in both MySQL Cluster setting and the HBase setting. We also show that Rameter does not suffer from such problem. In MySQL Cluster experiments, we mainly focus on the CPU resource accounting. We show results for the accounting of the most bottlenecked resource for the shared service, which we find to be CPU cycles for MySQL and network bandwidth for HBase. One interesting power of Rameter is the capability to control the resource usage at the thread-level in real-time, transparently to the guest VMs. We demonstrate our early implementation of this functionality by showing how it can be used in systems management tasks. In HBase experiments, we put more emphasis on the accounting of network I/O bandwidth. Also, in order to provide the proof of Rameter’s capability to perform resource accounting at any node within the shared service infrastructure, we present the accounting results of non-front-end node of HBase settings.

7.3.1 Clustered MySQL as the Shared Service

Experimental setup. Fig. 6 shows the set-up of our MySQL cluster that is used as a shared service by three CEs. Two of these CEs use the TPC-W benchmark [29] to generate workload for the database, while the third CE uses RUBiS [30]. The cluster consists of a front-end SQL node that interacts with the CEs, three data nodes, and a management node; each node is hosted within its dedicated server. One interesting aspect of the cluster’s operation is that even in the absence of any workload imposed by the CEs, a large

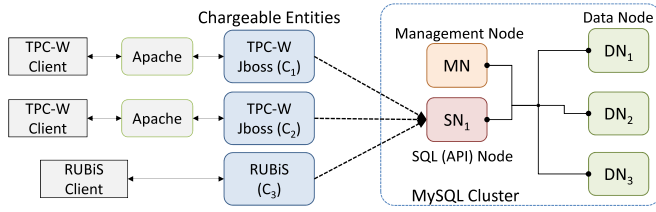


Fig. 6. Shared MySQL cluster setting. Three CEs labeled C_1 , C_2 , and C_3 share this database service.

number of small messages are exchanged between all pairs of nodes within the cluster for liveness check. The CEs house separate/non-overlapping data within the database which is spread across the three data nodes, and the cluster has a replication degree of 1.

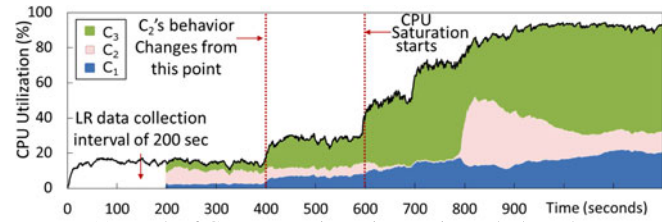
Experiment design and key findings. Given exact accuracy numbers are elusive, we compare the efficacy of Rameter and LR in the following online resource control situation: we wish to ensure that when the aggregate workload imposed upon the MySQL cluster causes its server CPUs to saturate, we identify the contribution of various CEs to this “overload,” and then enforce targeted CPU throttling only to the CE causing the overload. We implement a CPU throttling mechanism within the Xen hypervisors of the MySQL cluster servers which manipulates the rate at which timer interrupts are delivered to the guest VM only when the thread serving the CE causing the overload is to be scheduled.

We configure CEs to impose a dynamically changing workload (consisting of three phases) on MySQL as described in Table 2. In phase 1, all CEs generate a low-intensity workload, whose aggregate does not saturate the MySQL servers. During phase 2, starting at $t = 400$ s, C_2 starts to issue CPU-intensive requests. We are interested in observing how LR and Rameter handle this sudden change of behavior. Finally, in phase 3, starting at $t = 600$ s, C_2 issues continually increasing workloads that cause the CPU to saturate. Here we are interested in observing how our simple resource throttling performs based on the accounting information offered by LR and Rameter.

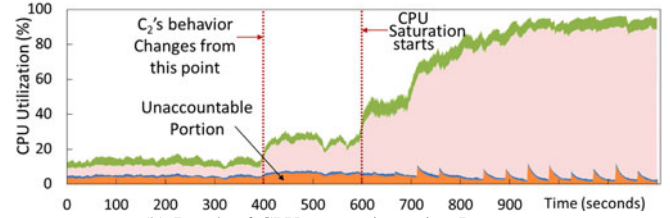
Since we do not have precise knowledge (i.e., ground truth) about true resource consumption, we engineer the workloads so that the CPU consumption imposed by the CEs is significantly different from each other, allowing us to rank their contributions without ambiguity. For example, we make the CPU consumption of C_2 much larger than others starting at $t = 400$ s so that other CEs cannot be mistaken as heavy CPU consumers. We begin by taking an in-depth look at the CPU accounting information offered by

TABLE 2
Workload Playback Scenario

Phase	Time Window	Workload	Top User
Phase 1	0-400 s	All 3 CEs generate light loads	C_2
Phase 2	400-600 s	C_2 starts to issue CPU-heavy requests C_2 's workload overwhelms	C_2
Phase 3	600-1,200 s	CPU, load increases every 100 s	C_2



(a) Result of CPU accounting using LR in stacked graph.



(b) Result of CPU accounting using Rameter.

Fig. 7. Comparison of CPU accounting results. CPU usage of MySQL Cluster SQL node is being accounted. By comparing the areas of equivalent color we see the rank order determined by each technique as well as accuracies.

LR and Rameter at one of the MySQL servers, SN, and how it evolves during phases 1-3 (results for CPU accounting of other nodes are qualitatively similar and we do not present them in the interest of space).

Figs. 7a, 7b show CPU accounting for the SQL node (SN) as carried out by LR and Rameter, respectively. We use a “stacked” representation, where the area under the curve corresponding to a CE represents the CPU usage charged to it. During phase 1, both LR and Rameter produce correct rank orders of CEs, although LR slightly overestimates the CPU consumption for C_2 . However, during phase 2, LR starts to report incorrect rank order: it determines C_3 to be the cause of the increased CPU usage. Upon investigating the reason for this mistake by LR, we find the following. While C_2 issues CPU-heavy requests and waits for MySQL’s response, the CPU utilization stays at high level. During this, C_3 continues to issue requests at a relatively high rate that are not CPU-heavy. However, the higher rate of requests coming from C_3 causes LR to infer spurious positive correlation between C_3 ’s requests and SN’s CPU usage. In fact, LR is unable to correct this throughout phase 2.

As we show in Fig. 7b, besides correctly identifying the correct rank order in its accounting, Rameter also reports what portion of the CPU usage of SN’s server it finds *unaccountable*. This amount indicates that Rameter’s algorithm was unable to charge the given thread’s resource usage to any of the chargeable entities because no direct association was found. This can happen if some thread is spawned independently of input requests from the chargeable entities and performs maintenance jobs. Or, it could be due to the nature of the thread that is created to service other running threads. In any case, Rameter provides this resource usage to the user and it is up to the user to divide up among chargeable entities. The most reasonable division would be to divide the ‘unaccountable’ portion according to the proportion of resource usage by each chargeable entity within that time window.

Fig. 9 quantifies the accuracy of accounting results of Fig. 7. In order to get the ground truth, we perform separate individual runs and use the CPU measurements as

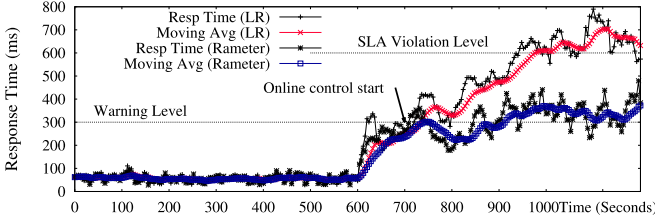


Fig. 8. Response time development of RUBiS for two cases—throttled by LR, and controlled by Rameter. LR incorrectly picks C_3 as a culprit for performance degradation. Throttling the request rate of C_3 has no effect. However, Rameter is able to prevent SLA violation.

estimations of true consumption (labeled as ‘separate runs’). Comparing the SUM columns, we observe that combined workload generates less CPU consumption than the sum of individual runs in our application. Therefore, we use the percentage of totals as the basis for comparison. Fig. 9b shows the average difference of the percentage of totals for Rameter and LR when compared to the ‘Separate Runs’. Rameter consistently differs by 4.3–5.3 percent whereas LR deviates by larger amount as workload intensifies from phase 1 to phase 3.

During phase 3, starting at $t = 600$ s, C_2 starts to saturate the CPU by drastically increasing the workload it imposes as described in Table 2. As portions of Figs. 7a and 7b for this phase show, LR continues to perform incorrect accounting. This has a detrimental effect on our CPU policing based resource control. Fig. 8 shows the change of response time for C_3 whose RUBiS application is accessing the shared MySQL Cluster service. Starting from time 600, the response time increases. We have set the response time of 300 ms as the initial warning level and 600 ms as the SLA violation level. The CPU saturation caused by C_2 continues to degrade the response time of RUBiS and eventually it violates the SLA. Since LR determines that C_3 , not C_2 , is the source of overload (see Fig. 7a), C_2 is not marked for any counter actions. However, Rameter is able to identify true cause of the overload and, starting at time 730, it initiates the CPU throttling for C_2 . Fig. 8 indicates that the moving average of response time under the control of Rameter is able to contain the response time below the SLA limit. This demonstrates one promising capability of Rameter (i.e., the thread-level monitoring technique) in critical resource managements of such shared resources.

7.3.2 HBase as the Shared Service

Experimental setup. Our second real-world shared service is HBase, a key-value storage system offering an open-source implementation of Google’s Bigtable, that has significantly different resource usage characteristics from a database such as MySQL. An HBase cluster consists of “region

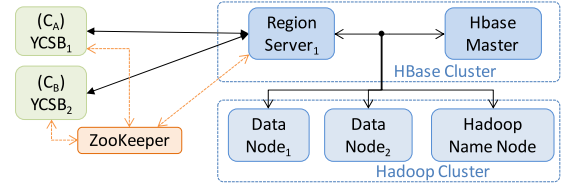


Fig. 10. Set-up of HBase as a shared service. We have two CEs labels as YCSB₁ and YCSB₂ running YCSB workload generator.

servers” and the HBase “master” that manages these region servers. The region servers act as in-memory caches for the contents of the data nodes. HBase stores its data in a Hadoop cluster, and this is a great example of a shared service (HBase) relying upon another shared service (Hadoop) to cater to the needs of CEs using it. Due to space constraints, we only discuss accounting of HBase servers. A region server employs an HDFS client to communicate with the Hadoop cluster that stores data persistently. HBase employs Zookeeper for coordinating distributed operations and locating region servers. We configure our HBase with a single region server. HBase operation involves significant data transfer from the data nodes to the region server, whereas the CPU load imposed by most requests is small as most requests are for simple data retrieval or inversions. Since the network bandwidth available to the region server becomes the bottleneck resource well before the CPU, we highlight accounting results for network bandwidth. Our HBase caters to requests from two CEs derived from the YCSB workload generator [31] (C_A and C_B).

Experiment design and key findings. Fig. 10 is the configuration of our HBase installation used in these experiments. We run an experiment lasting 500 seconds, during which the loads offered by C_A and C_B are varied as follows: (i) during $t = 0$ to $t = 100$ s, both C_A and C_B generates identical workloads which contains 5 percent update requests, (ii) at $t = 100$ s, C_B changes to a read-intensive mode with good temporal locality, which incurs high hits in the region server causing its CPU usage to increase proportionally with the network traffic sent to C_B , (iii) at $t = 200$ s, C_A starts to issue CPU-intensive insert-type requests that cause the CPU usage at the region server to increase. Fig. 11 shows the network traffic size inbound to the region server from the two CEs.

Fig. 12a and 12b shows the result of accounting the network bandwidth by LR and Rameter at the region server of HBase. The inputs to the LR are two time series of inbound network traffic from two chargeable entities as shown in Fig. 11. We have configured LR to use 100 second-long data as an input length in this HBase’s resource accounting, producing no accounting results for the first 100 seconds of the

	Phase 1(200-400sec)				Phase 2(400-600sec)				Phase 3(600-1200sec)			
	C_1	C_2	C_3	SUM	C_1	C_2	C_3	SUM	C_1	C_2	C_3	SUM
Separate Runs	2.8	8.1	7.1	17.9	2.8	22.2	6.7	31.6	2.8	74.9	13.0	90.6
	15.5%	45.0%	39.5%	100.0%	8.7%	70.1%	21.2%	100.0%	3.0%	82.6%	14.3%	100.0%
Rameter	1.5	7.9	5.5	14.8	1.6	21.4	5.0	28.0	1.1	74.0	7.3	82.4
	10.1%	53.0%	36.9%	100.0%	5.7%	76.5%	17.8%	100.0%	1.4%	89.8%	8.9%	100.0%
LR	2.6	7.7	4.6	14.9	7.1	5.1	16.5	28.7	15.9	14.4	50.3	80.6
	17.2%	51.8%	31.0%	100.0%	24.7%	17.8%	57.5%	100.0%	19.7%	17.9%	62.4%	100.0%

(a) CPU Utilization per CE and percentage of totals

	C_1	C_2	C_3	AVG	C_1	C_2	C_3	AVG	C_1	C_2	C_3	AVG
Rameter	5.4%	8.0%	2.6%	5.3%	3.0%	6.5%	3.4%	4.3%	1.7%	7.1%	5.5%	4.7%
LR	1.7%	6.8%	8.5%	5.6%	15.9%	52.2%	36.3%	34.8%	16.7%	64.7%	48.1%	43.2%

(b) Comparison of percentage of totals

Fig. 9. Comparison of accounting accuracy.

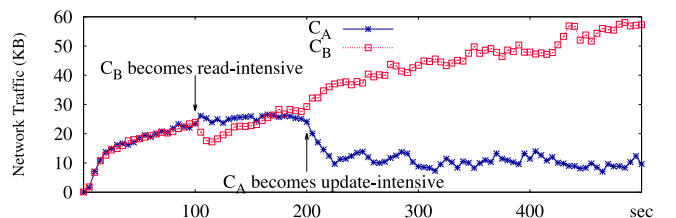


Fig. 11. Evolution of incoming network traffic to the region server from two CEs. Both C_A and C_B sends similar requests to HBase during $t = 0$ to $t = 100$ s consuming equal network bandwidth. C_A changes its behavior at $t = 100$ s, and C_B changes its behavior at $t = 200$ s.

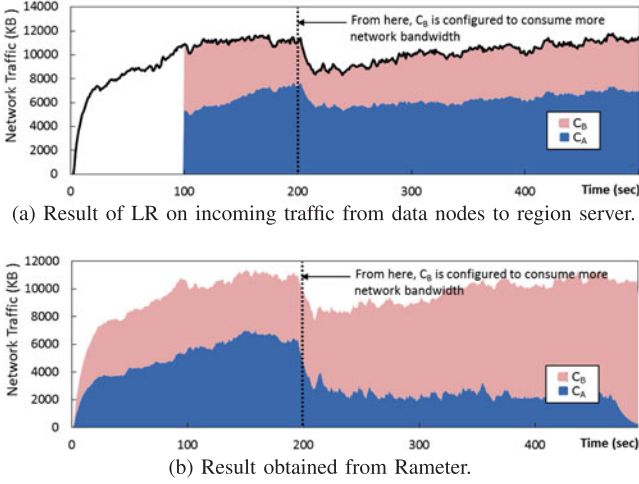


Fig. 12. Comparison of accounting results between LR and Rameter on the out-bound network traffic from data node to the region server.

run. The accounting results from Rameter are presented in Fig. 12b. In both of the stacked graphs, the upper area corresponds to the portion of network bandwidth used by C_B and the lower one, the portion used by C_A . The network bandwidth usage of C_A drops at $t = 200$, thus making C_B the heavier consumer of network bandwidth. LR continues to report C_A as the dominant consumer of network bandwidth. In contrast, Rameter correctly reflects this resource usage by CEs. (See Fig. 12b after $t = 200$ s). The misjudgment by LR can be explained in terms of caching effects. After $t = 200$, the traffic from C_B to the region server doubles by the end of the run (See Fig. 11) whereas the traffic from data nodes increases by only 20 percent. We believe this is due to caching within the region server and HBase documentation supports this conjecture.

We also present some of the selected accounting results using Rameter. Fig. 13a shows the CPU accounting at the region server for C_A and C_B . According to our preplanned workload scenario, C_B should consume more CPU than C_A after time 200 sec. The accounting result indicates this behavior. Notice the significant portion of unaccountable CPU usage at the bottom region of the stacked graph

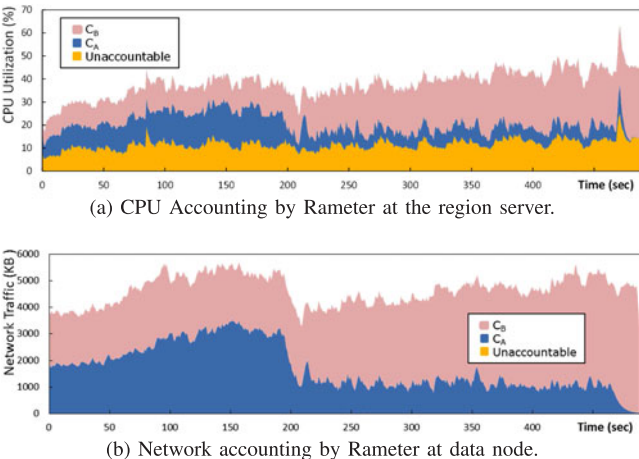


Fig. 13. Results by Rameter at various nodes of HBase. (b) shows Rameter's capability to account resource at multiple hops away from the front-end of the shared service. Note that data node of HBase does not have a direct contact with CEs.

TABLE 3
Overhead of System call Interception and Information Recording

System call name	Typical Avg Turn-around time	System call Interception	System call Intcpt&Record
read	72.0 microsec	.38%	.59%
recv	39.6 microsec	.71%	1.07%
write	26.4 microsec	1.07%	1.60%
send	134.3 microsec	.21%	.31%
open	41.9 microsec	.67%	1.01%
accept	14.6 microsec	1.93%	2.89%
connect	148.0 microsec	0.19%	0.29%
pread	54.0 microsec	0.52%	0.78%

(orange color). It includes CPU cycles consumed for communicating with the Zookeeper, HBase master and Hadoop Namenodes. Especially, we have noticed two high peaks at around 80 and 470 sec. By studying the HBase documentations, we have concluded that these would most likely be due to the I/O compaction at the region server.

7.4 Overhead

The overhead of *Rameter* exists at both the local monitoring and the collective inference steps. During the local monitoring step, the control of the intercepted system call is delivered to the hypervisor and several information including system call parameters, time stamps and thread identifiers are recorded. Since these tasks must be completed before returning the control back to the system call handler of VM's guest kernel, the delay here directly affects the performance. Therefore, our overhead measurement closely focuses on this aspect. However, we consider the overhead of collective inference to be negligible. For the offline mode, the overhead of inference is out of the critical path since it is performed at separate server node. For the online mode needed by the resource control, the hypervisor manages small number of variables corresponding to the CEs and carries out simple arithmetic. Thus, we present the overhead of the local monitoring step below.

In order to better explain the overhead of *Rameter's* local monitoring step, we provide our measurement data in Table 3. It shows the performance impact of two core mechanisms to various system calls. The first mechanism is the system call interception alone within the hypervisor. The second mechanism is the system call interception and data recording (via TRACE_xD macro of xen). We first measured pure time overhead of these two operations per individual system calls through repeated measurements of a dummy system call that had empty logic inside the function. System call interception alone turns out to add 282 nanosec of overhead to each system call. When data recording step is combined, total of 422 nanosec is incurred per each system call. The second column of Table 3 provides our measurements of typical system call turn-around times. Compared to those turn-around times, the overhead of system call interception and data recording is only about 1-2 percent and the worst case is less than 3 percent. Therefore, the data gathering during the local monitoring step incurs small overhead to the running applications within the guest VM.

8 RELATED WORK

Earlier works by Banga et al. have addressed the issue of resource accounting within a single host [10]. They introduced new abstraction, called *resource containers*, to be used as a new resource principal within the kernel. Realization of the *resource container* required modifications of the kernel as well as applications. Distributed resource container [21], and Power Container [32] are extensions of it to the distributed environment in which local resource containers are bound together by exchanging global identifiers via IPv6 optional field in order to coordinate the resource consumption across hosts. The goal is to throttle the energy consumption per applications. The use of resource container relies on the help from the application for correct and timely binding of resource principals to the container, although they also describe heuristics to automatically detect the bindings. The OS has to be modified to support the necessary data structure for representing the resource containers (and the propagation of it to other hosts in case of the distributed resource container).

The resource accounting problem has been studied by a number of researchers. Reumann et al. [7] have recognized the problem and proposed a mechanism, *Stateful Distributed Interposition*, for sharing the application context information across multi-tiered servers to support actions such as enforcing resource quota. They defined the context abstraction that included the identity of the client and achieved the creation of the contexts and propagation of them across server via modification of OS and applications. Users are required to recompile the application after instrumenting it. Fonseca et al. [8] described an architecture, *Quanto*, for tracking the energy consumption of embedded devices. To achieve the goal of energy tracking, they have developed a framework for resource accounting of distributed devices. Since their target environment is embedded systems, they were able to support the scenario in which they make modifications to the OS kernel and require the developers to write applications that notify the OS of the owner of various activities. In one of the most recent work by Narasayya et al. [9], the resource accounting problem has been addressed in the context of mitigating the performance interferences due to resource sharing within SQL Azure RDBMS. Their accounting mechanism relies on modifying the internals of the SQL Azure database.

There are plenty of works related to monitoring of distributed systems that are relevant to the resource accounting although they do not explicitly target the resource accounting problem we address in this work. Ganglia [33] is a distributed monitoring system with scalability in mind. The goal is to provide statistics related to various resource types per server in clusters. It does not support fine-grained resource usage monitoring per chargeable entities. Chopstix [34] provides detailed monitoring information about low-level OS events. It adds a data structure, *sketches*, to the kernel in order to monitor page allocation, mutex/semaphore locking, and CPU utilization. They focus on building a sampling-based system that monitors events of interest as a high resolution with low overhead. OProfile [35] is capable of delivering detailed system-wide profile information. It provides convenient access to the performance counters. Nagios [36] is an open source monitoring system that monitors CPU, disk,

logs as well as network usage of distributed applications. Rameter can benefit from such monitoring tools by extending it to leverage the information provided by them.

In the evaluation we have demonstrated a use of resource accounting in preventing SLA violation via our resource control technique. The problem of resource control has been investigated in several studies and various effective techniques have been developed [37], [38], [39], [40], [41]. One main difference of our resource control problem is that we are required to control the resource usage at finer thread granularity within a shared instance, whereas these studies aim to better-schedule separate VM instances. However, control theoretic techniques such as the one in Q-clouds [40] can be applied to ours to achieve accurate CPU usage ratio among CEs.

9 CONCLUSION

In this paper we have presented our study on the problem of resource accounting within an IT platform that offers shared services. In order to understand the nature of the problem and find effective solution we have explored two different accounting techniques built with different balance of emphasis - LR-based technique and *Rameter*. Our main contribution is the resource accounting framework, *Rameter*, that operates at the hypervisor. It combines the monitoring unit that collects fine-grained thread-level data and the inference unit that applies light-weight inference. Comparing with LR-based approach, our analysis revealed that there could be cases where more fine-grained monitoring information does not only provide better accuracy, but also impact critical management decisions. *Rameter* also incurs only about 1-2% of the turn-around time overhead to the system calls that are intercepted.

REFERENCES

- [1] E. M. Haber, E. Kandogan, and P. Maglio, "Collaboration in system administration," *Queue*, vol. 8, no. 12, pp. 10:10–10:20, Dec. 2010.
- [2] E. Kotsovinos, "Virtualization: Blessing or curse?" *Queue*, vol. 8, pp. 40:40–40:46, 2010.
- [3] Amazon SimpleDB. [Online]. Available: <http://aws.amazon.com/simplydb/>, 2015.
- [4] SQL Azure Whitepaper. [Online]. Available: <http://social.technet.microsoft.com/wiki/contents/articles/1695.inside-windows-azure-sql-database.aspx>, 2010.
- [5] Latency is everywhere and it costs you sales-how to crush it. [Online]. Available: <http://highscalability.com/blog/2009/7/25/latency-is-everywhere-and-it-costs-you-sales-how-to-crush-it.html>, 2009.
- [6] S. Agarwala, F. Alegre, K. Schwan, and J. Mehalingham, "E2eprof: Automated end-to-end performance management for enterprise systems," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 749–758.
- [7] J. Reumann and K. G. Shin, "Stateful distributed interposition," *ACM Trans. Comput. Syst.*, vol. 22, no. 1, pp. 1–48, Feb. 2004.
- [8] R. Fonseca, P. Dutta, P. Lewis, and I. Stoica, "Quanto: Tracking energy in networked embedded systems," in *Proc. 8th USENIX Conf. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 2008, pp. 323–338.
- [9] V. R. Narasayya, S. Das, M. Syamala, B. Chandramouli, and S. Chaudhuri, "Sqlvm: Performance isolation in Multi-tenant relational Database-as-a-service," in *Proc. CIDR*, 2013, pp. 1–9.
- [10] G. Banga, P. Druschel, and J. C. Mogul, "Resource containers: A new facility for resource management in server systems," in *Proc. 3rd Symp. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 1999, pp. 45–58.

- [11] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. Katz, S. Shenker, and I. Stoica, "Mesos: A platform for Fine-grained resource sharing in the data center," in *Proc. 8th USENIX Conf. Netw. Syst. Des. Implementation*, Berkeley, CA, USA, 2011, pp. 295–308.
- [12] Mesosphere. [Online]. Available: <https://mesosphere.com/>, 2015.
- [13] Stratoscale. [Online]. Available: <http://www.stratoscale.com/>, 2015.
- [14] D. Wentzlaff and A. Agarwal, "Factored operating systems (FOS): The case for a scalable operating system for multicores," *SIGOPS Oper. Syst. Rev.*, vol. 43, no. 2, pp. 76–85, Apr. 2009.
- [15] Openstack. [Online]. Available: <http://www.openstack.org/>, 2015.
- [16] (2008). The Force.com Multitenant Architecture: Understanding the Design of Salesforce.com's Internet Application Development Platform. [Online]. Available: in *Force.com Whitepaper* http://wiki.developerforce.com/index.php/Multi_Tenant_Architecture
- [17] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, "Bigtable: A distributed storage system for structured data," in *Proc. 7th USENIX Symp. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 2006.
- [18] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proc. 7th Symp. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 2006, pp. 335–350.
- [19] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The google file system," in *Proc. 19th ACM Symp. Operating Syst. Principles*, New York, NY, USA, 2003, pp. 29–43.
- [20] M. Y. Chen, A. Accardi, E. Kiciman, J. Lloyd, D. Patterson, A. Fox, and E. Brewer, "Path-based failure and evolution management," in *Proc. 1st Conf. Netw. Syst Des Implementation*, Berkeley, CA, USA, 2004, pp. 23–23.
- [21] A. Weissel and F. Bellosa, "Dynamic thermal management for distributed systems," in *Proc. 1st Workshop Temperature-Aware Comput. Syst.*, Munich, Germany, Jun. 2004, pp. 1–11.
- [22] P. Barham, A. Donnelly, R. Isaacs, and R. Mortier, "Using magpie for request extraction and workload modelling," in *Proc. 6th Conf. Symp. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 2004, pp. 18–18.
- [23] A. Dinaburg, P. Royal, M. Sharif, and W. Lee, "Ether: Malware analysis via hardware virtualization extensions," in *Proc. 15th ACM Conf. Comput. Commun. Security*, New York, NY, USA, 2008, pp. 51–62.
- [24] D. Gupta, L. Cherkasova, R. Gardner, and A. Vahdat, "Enforcing performance isolation across virtual machines in xen," in *Proc. ACM/IFIP/USENIX Int. Conf. Middleware*, 2006, pp. 342–362.
- [25] T. Wood, L. Cherkasova, K. Ozonat, and P. Shenoy, "Profiling and modeling resource usage of virtualized applications," in *Proc. 9th ACM/IFIP/USENIX Int. Conf. Middleware*, 2008, pp. 366–387.
- [26] A. Kansal, F. Zhao, J. Liu, N. Kothari, and A. A. Bhattacharya, "Virtual machine power metering and provisioning," in *Proc. 1st ACM Symp. Cloud Comput.*, New York, NY, USA, 2010, pp. 39–50.
- [27] J. C. McCullough, Y. Agarwal, J. Chandrashekar, S. Kuppuswamy, A. C. Snoeren, and R. K. Gupta, "Evaluating the effectiveness of Model-based power characterization," in *Proc. USENIX Conf. USENIX Annu. Tech. Conf.*, Berkeley, CA, USA, 2011, p. 12.
- [28] C. Stewart, T. Kelly, and A. Zhang, "Exploiting nonstationarity for performance prediction," in *Proc. 2nd ACM SIGOPS/EuroSys Eur. Conf. Comput. Syst.*, 2007, pp. 31–44.
- [29] W. Smith. TPC-W: Benchmarking An Ecommerce Solution. [Online]. Available: <http://www.tpc.org/information/other/techarticles.asp>, 2010.
- [30] RUBiS. [Online]. Available: <http://rubis.objectweb.org/>, 2008.
- [31] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proc. 1st ACM Symp. Cloud Comput.*, New York, NY, USA, 2010, pp. 143–154.
- [32] K. Shen, A. Shriraman, S. Dwarkadas, X. Zhang, and Z. Chen, "Power containers: An os facility for fine-grained power and energy management on multicore servers," in *Proc. 18th Int. Conf. Archit. Support Program. Lang. Operating Syst.*, 2013, pp. 65–76.
- [33] M. L. Massie, B. N. Chun, and D. E. Culler, "The ganglia distributed monitoring system: Design, implementation and experience," *Parallel Comput.*, vol. 30, pp. 817–840, 2003.
- [34] S. Bhatia, A. Kumar, M. E. Fiuczynski, and L. Peterson, "Lightweight, High-resolution monitoring for troubleshooting production systems," in *Proc. 8th USENIX Conf. Operating Syst. Des. Implementation*, Berkeley, CA, USA, 2008, pp. 103–116.
- [35] John Levon. Oprofile. [Online]. Available: <http://oprofile.sourceforge.net/credits/>, 2014.
- [36] Introducing the Windows Azure Platform. [Online]. Available: <http://go.microsoft.com/?linkid=9752185>, 2011.
- [37] F. Checconi, G. Anastasi, and L. Abeni, "Respecting temporal constraints in virtualised services," in *Proc. 2nd IEEE Int. Workshop Real-Time Serv.-Oriented Archit. Appl.*, 2009, pp. 73–78.
- [38] F. Checconi, T. Cucinotta, D. Faggioli, and G. Lipari, "Hierarchical multiprocessor cpu reservations for the linux kernel," in *Proc. 5th Int. Workshop Operating Syst. Platforms Embedded Real-Time Appl.*, 2009, pp. 15–22.
- [39] J. Lee, S. Xi, S. Chen, L. T. X. Phan, C. Gill, I. Lee, C. Lu, and O. Sokolsky, "Realizing compositional scheduling through virtualization," in *Proc. IEEE 18th Real Time Embedded Technol. Appl. Symp.*, Washington, DC, USA, 2012, pp. 13–22.
- [40] R. Nathuji, A. Kansal, and A. Ghaffarkhah, "Q-clouds: Managing performance interference effects for qos-aware clouds," in *Proc. 5th Eur. Conf. Comput. Syst.*, New York, NY, USA, 2010, pp. 237–250.
- [41] S. Xi, J. Wilson, C. Lu, and C. Gill, "Rt-xen: Towards Real-time hypervisor scheduling in xen," in *Proc. 9th ACM Int. Conf. Embedded Softw.*, New York, NY, USA, 2011, pp. 39–48.



Byung Chul Tak is a research staff member at IBM T.J. Watson Research Center, Yorktown Heights, NY. He received his PhD in computer science in 2012 from Pennsylvania State University. He received his MS degree in computer science from Korea Advanced Institute of Science and Technology (KAIST) in 2003, and his BS from Yonsei University, Korea in 2000. Prior to joining Pennsylvania State University, he worked as a researcher in the Electronics and Telecommunications Research Institute (ETRI), Daejeon, Korea. His research interest includes virtualization, operating systems and cloud computing.



Youngjin Kwon is a PhD candidate at computer science department, the University of Texas at Austin. He received his BS degree in computer science from Sogang University in 2007, and his MS degree in computer science from Korea Advanced Institute of Science and Technology (KAIST) in 2009. He started his PhD program in Sep. 2012. His current research interests are in virtualization technology, improving operating system, and system security.



Bhuvan Urgaonkar is an associate professor in the department of computer science and engineering at the Pennsylvania State University. He received his MS (2002) and PhD (2005) degrees in computer science at the University of Massachusetts, and his BTech (1999) in computer science and engineering at IIT Kharagpur. He is a recipient of the NSF CAREER Award, research awards from HP Labs and Cisco, and has co-authored best student papers at IEEE MASCOTS 2008 and ICAC 2005 conferences. His research involves applying ideas from distributed computing, resource management, scheduling, performance evaluation, and analytical modeling to the design and evaluation of data centers, networked systems, operating systems, virtualization techniques, and storage systems. His current research focus is in cloud computing, power management of data centers, and storage systems. Urgaonkar is a senior member of IEEE and senior member of ACM.